

Team Members: Nicole Linares

Project Title: AADR-Subset: A Python Package for Filtering and Exporting the Allen Ancient DNA Resource

Project Type: Software Package/Tool

Tool/Package Name: aadrkit

1. **Problem Statement:** Researchers working with the Allen Ancient DNA Resource (AADR) - a centralized dataset of over 17,000 ancient individuals and over 6,000 modern individuals for scientists to produce analyses of population history and natural selection - must repeatedly extract project-specific population subsets (including by region, time period, lineage, or coverage) before running analyses. Currently, this requires writing ad hoc, one-off scripts to parse EIGENSTRAT files for every new research question, as no lightweight, standardized, reusable tool exists to generate these subsets directly from AADR's per-individual metadata without first manually translating region, date, or coverage criteria into population lists, which is both inconvenient and a reproducibility risk. This project builds a Python package that automates AADR filtering and re-exports these subsets into standard formats (EIGENSTRAT - the population structure format for the EIGENSOFT software developed by the David Reich Lab /PLINK - the open-source genome-wide association format for industry-standard general bioinformatics use, quality control, and GWAS; used by AADR) usable by downstream tools like EIGENSOFT (population genetics structure software package) and ADMIXTOOLS (population genetics statistical testing software package), reducing the need for repeated manual scripting with a single reusable command.
2. **Data Source(s):** The primary data source is the Allen Ancient DNA Resource (AADR), which is maintained by the Reich Lab at Harvard and hosted on the Harvard Dataverse (DOI: 10.7910/DVN/FFIDCW) at its current version v66.0, with data from over 17,000 ancient and 6,000 modern individuals. AADR is freely and publicly available and downloadable with *no access request required*. Use of this database requires two citation obligations: 1) the AADR paper itself (with the specific version DOI used), and 2) the original publication(s) with any individual samples included in a derived analysis, since AADR is a *compilation* of previously published data rather than the original source. AADR's genotype files are distributed in EIGENSTRAT format, which requires the EIGENSOFT toolkit (specifically the `convertf` utility - a tool that formats and translates genomic data between different software styles) to parse and convert the data. Without it, the raw data is not directly usable. EIGENSOFT is open-source and freely available. Its role in this project is discussed further under "Planned methods and tools".
3. **Planned Methods and Tools:** This project builds *aadrkit*, a Python package that filters the Allen Ancient DNA Resource (AADR) by user-specified criteria (like region, time period, population, coverage) and re-exports the result in EIGENSTRAT or PLINK format. The package will: 1) parse AADR's `.anno` metadata file using pandas to identify

individuals matching filter criteria, 2) construct population/individual inclusion parameters required by EIGENSOFT's ``convertf`` utility, 3) implement ``convertf`` via Python's ``subprocess`` module to perform the underlying extraction and format conversion, and 4) return a ready-to-use subset. The package will be structured as an installable Python package (pyproject.toml- a standardized configuration file used for Python projects that allows easy setup and usage of a Python tool/project) with a command-line interface, a pytest suite validating filter logic against known AADR records, and dependency management via a conda environment file (EIGENSOFT is most reliably installed through conda-forge/bioconda). Development is tracked with Git/GitHub throughout. A JSON export option (tying to Week 9's coverage of APIs/JSON) was considered but ultimately excluded, since AADR's ``anno`` metadata is flat and tabular. JSON offers no structural or functional advantage over the CSV/TSV exports researchers already expect. Adding it would just serve to align the project with the syllabus rather than genuine utility.

- a. Course concept mapping for Weeks 1-10:
 - i. **Week 1: Computing Environments** - shell/``subprocess``-based implementation of external CLI tool (``convertf``)
 - ii. **Week 2: Reproducible Research Fundamentals** - Conda-based dependency management for a package with a non-Python external dependency
 - iii. **Week 4: Text Processing** - Parsing/filtering the semi-structured ``anno`` metadata file
 - iv. **Week 5: Data Wrangling** - Building a samples x metadata table in pandas from ``anno``
 - v. **Week 6: Software Design** - Modular design (parsing/filtering/export separated) + version control
 - vi. **Week 7: Advanced Software Design** - Python packaging structure, AI-assisted development/debugging
4. **Generative AI Plan:** AI assistance will be used across several concrete stages of this project: 1) generating setup code for the ``subprocess`` wrapper around ``convertf``, the pandas-based ``anno`` filtering functions, and the ``pytest`` test scaffolding, 2) debugging environment and dependency issues (like conda/bioconda setup and package conflicts) and ``convertf`` setup file formatting errors, 3) drafting docstrings for the code, and 4) interpreting third-party tool documentation (EIGENSOFT/``convertf`` usage, bioconda packaging conventions) where official documentation is sparse or assumes prior familiarity. AI output will be validated in ways specific to each use case, not just reviewed by eye. Code generated or modified with AI assistance is checked against the ``pytest`` suite comparing filter/export results to manually verified AADR records (see Planned Methods and Tools). AI explanations of external tool behavior (like ``convertf`` flags, bioconda conventions, etc.) are cross-checked against the tools' own

documentation or source before being relied on in the design. Debugging suggestions are only adopted after being tested against the actual failure, not assumed as correct. All AI usage will be documented per course policy - model name and version, key prompts used, and what was accepted, corrected, or rejected - in an `AI_USAGE.md` file maintained alongside the code.

5. **Team Roles:** This project is completed individually, with prior instructor approval for a solo project. As the sole contributor, Nicole Linares (myself) is responsible for all aspects of the work, including acquiring and validating AADR data access, designing and implementing the `aadrkit` package (metadata filtering, `convertf` integration, CLI, and export logic, writing and maintaining the test suite, documenting and validating AI-assisted output per course policy, and preparing the final report and presentation. Relevant preparation for this scope is already evident in this repository's existing reproducible environment setup (conda, Docker, version control from Lab 1), which will be extended to support this project's additional dependencies.
6. **Timeline:**
 - a. **Oct 7th - Oct 21st (Weeks 8-9):** Light setup - install EIGENSOFT via bioconda, obtain small AADR test subset, manually validate one `convertf` extraction from the command line before writing any wrapper code
 - b. **Oct 21st - Oct 28th (Week 10):** Prototype `.anno` parsing and filter functions in pandas (region/date/coverage)
 - c. **Nov 2nd - Nov 4th (Week 11):** Build the `subprocess` based `convertf` wrapper, connect filtering output to extraction and have first working end-to-end prototype
 - d. **Nov 9th - Nov 11th (Week 12):** Formalize package structure (`pyproject.toml`, CLI) and write `pytest` suite validating filters against known AADR records
 - e. **Nov 16th - Nov 18th (Week 13):** Documentation (README, docstrings, AI_USAGE) and handle edge cases surfaced by testing
 - f. **Nov 23rd - Nov 27th:** Thanksgiving Break
 - g. **Nov 30th - Dec 2nd (Week 14):** Final polish - confirm reproducibility (clean code + run-through and complete report draft)
 - h. **Dec 7th - Dec 9th (Week 15):** Finalize and rehearse presentation
7. **Risks/Open Questions:**
 - a. **Risk:** EIGENSOFT has no Windows build (bioconda channels are Linux/macOS-oriented)
 - i. **Mitigation:** Development and testing will occur within WSL2, which is already configured in this repository's environment set up from Lab 1. This avoids having to set up a new platform mid-project.
 - b. **Risk:** `convertf`'s parameter-file (setup-file) syntax and edge-case behavior are sparse

- i. **Mitigation:** Validate behavior empirically against small, manually verified test cases *before* building filtering logic on top of it. If needed, consult EIGENSOFT's source/GitHub issues where official documents fall short.
- c. **Risk:** With this being a solo project, there is no teammate to help meet schedule deadlines or divide up the workload
 - i. **Mitigation:** Front-load the technical validation (during Weeks 8 - 10) before full development begins. Additionally, there is a one-week buffer around Thanksgiving.
- d. **Risk:** AADR's most recent and current version is "v66.0," and a mid-semester release could change the file structures.
 - i. **Mitigation:** Pin development and testing to the *specific* version/DOI downloaded at the start of the project - no re-downloading newer versions mid project
- e. **Risk:** Establishing reliable "ground truth" for test cases requires carefully and manually verifying AADR records
 - i. **Mitigation:** Open question - the scope will be a small, manually checked set of individuals (by their ID) with known region/date/coverage values, rather than attempting a comprehensive validation across the full dataset